

Summer 2026 REU Code Cheat Sheet for Chris

Where every result is built, what to run, and which lines to show if asked

Repository: /Users/davidvalenta/deep-wind-model-dispatch | Frozen pushed commit: e184740da217ed811496c056aace5b7a61f6f860

How to use this during a meeting: Open the folder Chris asks about, run the matching RUN_*.py command, and use the line-number tables below to show where the forecast, Gurobi model, SoC update, revenue objective, and realized execution are actually implemented.

1. One-screen map

Project piece	Folder	Run command	Main thing it proves
Step 0 reference	Summer 2026 REU/100 MW baseload	<code>../venv/bin/python RUN_0_100MW_BASELOAD.py</code>	Builds the 100 MW constant-output storage reference and comparison tables.
Step 1 forecast	Summer 2026 REU/causal ridge regression	<code>../venv/bin/python RUN_1_FORECAST_RMSE.py</code>	Compares power-forecast RMSE; no Gurobi and no COVE here.
Step 2 deterministic dispatch	Summer 2026 REU/rolling horizon	<code>../venv/bin/python RUN_2_ROLLING_HORIZON.py</code>	Uses causal ridge forecasts, Gurobi, 24h execution, and horizon sweep.
Step 3 scenario dispatch	Summer 2026 REU/different scenarios	<code>../venv/bin/python RUN_3_SCENARIO_COMPARISON.py</code>	Tests 1/3/5/7/10 forecast futures and solves uncertainty-aware first-hour dispatch.
Step 4 oracle ceiling	Summer 2026 REU/oracle upper bound	<code>../venv/bin/python RUN_4_ORACLE_UPPER_BOUND.py</code>	Perfect-future upper bound: Gurobi sees realized future wind and price.
B6 verification	Summer 2026 REU/b6 verification	see B6_CANONICAL_RUNNER.py	Chris-specific 2020 package; separate from the paper ladder.

2. The ladder story

The paper-facing ladder is not one giant black box. It is a sequence: first prove which forecast is best, then feed that forecast into dispatch, then test whether multiple forecast futures help, then show the oracle upper bound.

Ladder level	What changes	What stays fixed	Metric
Baseload/reference	No optimization claim yet; creates reference delivery/revenue/COVE.	Same 100 MW / 10 h CAES setup when used as storage reference.	Revenue and COVE reference.
Causal ridge regression	Forecast method is selected by RMSE.	No dispatch yet. Same power data comparison.	RMSE, MAE, bias.
Rolling-horizon Gurobi	Gurobi uses the causal forecast and different lookahead horizons.	100 MW, 10 h, RTE 0.55, 249 MW grid cap, wind-only charging.	COVE and revenue vs baseload.
Scenario dispatch	Instead of one future forecast, Gurobi sees several plausible futures.	Same storage and grid constraints. First-hour decision only is executed.	Raw revenue and COVE vs baseload.
Oracle	Forecast is replaced by real future data.	Same Gurobi constraints; not realistic.	Upper-bound COVE improvement.

Important distinction: B6 is not the main paper experiment. B6 is the frozen 2020 verification request from Chris. The paper ladder is the organized Summer 2026 REU set.

3. Common storage and dispatch constraints

Constraint / parameter	Value or equation	Where to point
Storage power	100 MW	EXPERIMENT_KNOBS.py in rolling/scenario/oracle/baseload folders
Storage duration	10 hours	capacity = power x duration
Energy capacity	1000 MWh	100 MW x 10 h
RTE	0.55 for CAES	Passed through <code>--rte</code> or <code>util.get_rte(...)</code>
Grid export cap	249 MW	<code>--grid-cap-mw -> args.grid_cap_mw -> config["rated_capacity"]</code>
SoC min	20% of capacity = 200 MWh	<code>--min-soc-frac 0.2</code>
SoC max	100% of capacity = 1000 MWh	<code>--max-soc-frac 1.0</code>

Wind-only charging	direct + charge <= wind	rolling_horizon_gurobi_dispatch.py:146-148
Delivered power	delivered = direct + discharge	rolling_horizon_gurobi_dispatch.py:150-151
No simultaneous charge/discharge	binary mode u controls charge or discharge	rolling_horizon_gurobi_dispatch.py:153-155
Available energy	discharge / RTE <= SoC - min SoC	rolling_horizon_gurobi_dispatch.py:157-158
SoC update	SoC[t+1] = SoC[t] + charge - discharge/RTE	rolling_horizon_gurobi_dispatch.py:160-161
Objective	maximize sum(price[t] * delivered[t])	rolling_horizon_gurobi_dispatch.py:163

4. Step 0: 100 MW baseload folder

Purpose: build a constant-output storage reference. This is not a Gurobi revenue optimizer. It follows a rule: try to deliver 100 MW every hour using wind first, charge when wind is high, discharge when wind is low, and curtail leftovers.

Question Chris may ask	Answer	File and line
Where do I change baseload settings?	Edit DATA, storage power, duration, RTE, grid cap, target output.	100 MW baseload/EXPERIMENT_KNOBS.py:12-41
What does RUN_0 actually run?	It builds a command using EXPERIMENT_KNOBS and calls the worker scripts.	100 MW baseload/RUN_0_100MW_BASELOAD.py:89-119 and 122-145
Where is the 100 MW rule?	If wind >= 100, deliver 100 and charge extra; else deliver wind and discharge toward 100.	100 MW baseload/code/build_100mw_baseload_reference.py:98-119
Where is revenue calculated?	delivered * raw price and delivered * normalized price are stored hourly.	build_100mw_baseload_reference.py:121-140
Where is the hourly CSV written?	Writes constant_output_baseload_100mw_2014_2023_hourly.csv.	build_100mw_baseload_reference.py:311-314
Where are summaries made?	summarize_period computes raw revenue, normalized revenue, delivered energy, curtailment, SoC.	build_100mw_baseload_reference.py:146-169

5. Step 1: causal ridge regression folder

Purpose: choose the best short-term wind-power forecast. This step only compares prediction error. It does not run Gurobi, dispatch energy, or calculate COVE.

Part	What it does	File and line
Knobs	Chooses dataset, ridge alpha, old comparison file, output directory.	causal ridge regression/EXPERIMENT_KNOBS.py:12-27
Run command builder	Builds command for compare_forecast_rmse.py.	RUN_1_FORECAST_RMSE.py:49-68
Features	Builds speed, speed^2, speed^3, lagged power, and time features.	code/causal_lag_forecast.py:57-103
Ridge equation	Solves $(X^T X + \alpha I) w = X^T y$.	code/causal_lag_forecast.py:106-109
Speed-power curve baseline	Fits polynomial curve from wind speed to power.	code/causal_lag_forecast.py:112-119
Lag-1 persistence	Uses previous-hour power as prediction.	code/causal_lag_forecast.py:164
RMSE comparison	Compares causal lag/ridge, persistence, speed curve, RNN, physics, probabilistic predictions.	code/compare_forecast_rmse.py:90-124
Output table	Writes forecast_model_rmse_comparison.csv.	code/compare_forecast_rmse.py:121-124

If asked what causal means: The model only uses information available at that time: current wind speed, old power values, and calendar/time features. It is not allowed to peek at future actual power.

6. Step 2: rolling horizon, the important part

Purpose: take the causal forecast and let Gurobi decide when to send wind directly, charge storage, discharge storage, or curtail wind. The model solves a planning window, executes only the first 24 hours, carries the battery SoC forward, then solves again.

Flow step	What happens	File and line
Change settings	Edit storage, grid cap, horizons, direct reserve, output folder.	rolling horizon/EXPERIMENT_KNOBS.py:12-39
Build terminal command	RUN_2 turns knobs into flags like --grid-cap-mw and --horizons.	RUN_2_ROLLING_HORIZON.py:57-96
Parse flags	Worker receives those flags with argparse.	code/forecast_backtest_rolling_horizons.py:709-

		785
Build config	Storage type, power, duration, modules, and grid cap are inserted into config.	code/forecast_backtest_rolling_horizons.py:797-805
Normalize price	Raw LMP is capped and divided by training mean for this metric style.	code/forecast_backtest_rolling_horizons.py:814-828
Fit forecast models	Trains direct ridge models for future wind/price matrices.	code/forecast_backtest_rolling_horizons.py:112-166
Loop through horizons	For each origin, choose actual future if oracle or forecast future if causal.	code/forecast_backtest_rolling_horizons.py:382-418
Call Gurobi	solve_window receives planned generation, planned price, config, current SoC, constraints.	code/forecast_backtest_rolling_horizons.py:419-429
Apply direct reserve	For causal only, planned direct wind can be held higher to reduce under-forecast curtailment.	code/forecast_backtest_rolling_horizons.py:431-435 and 355
Execute realized operation	Uses actual wind for the executed 24 hours, clips charge/discharge/direct to feasible values.	code/forecast_backtest_rolling_horizons.py:436-443 and 235-298
Carry SoC forward	After execution, current SoC becomes the end SoC for the next solve.	code/forecast_backtest_rolling_horizons.py:466-470 plus subsequent rows
Check violations	Reports wind-only, grid, charge, discharge, SoC, and delivered-balance violations.	code/forecast_backtest_rolling_horizons.py:299-354

7. What Gurobi actually solves in rolling horizon

The actual MILP is in `rolling_horizon_gurobi_dispatch.py`, not in the RUN file. The RUN file only launches it indirectly through `forecast_backtest_rolling_horizons.py`.

Gurobi piece	Meaning	File and line
solve_window	Main optimization function: one planning window at a time.	rolling_horizon/code/rolling_horizon_gurobi_dispatch.py:101-111
Inputs	generation forecast/actual, price forecast/actual, config, initial SoC, terminal rule.	rolling_horizon_gurobi_dispatch.py:101-110
Storage rating/capacity	rating = storage_rating x modules; capacity = rating x duration.	rolling_horizon_gurobi_dispatch.py:115-121
Decision variables	P_dir, P_ch, P_dis, P_delivered, SoC, binary charge mode.	rolling_horizon_gurobi_dispatch.py:130-136
Initial SoC	Sets starting battery state for the window.	rolling_horizon_gurobi_dispatch.py:138
Terminal SoC	equal-initial, no-empty, or none.	rolling_horizon_gurobi_dispatch.py:139-144
Wind-only charging	P_dir + P_ch <= generation.	rolling_horizon_gurobi_dispatch.py:146-148
Delivered power	P_delivered = P_dir + P_dis.	rolling_horizon_gurobi_dispatch.py:150-151
Charge/discharge mode	Binary u prevents charging and discharging at same time.	rolling_horizon_gurobi_dispatch.py:153-155
Available energy	Discharge cannot pull battery below min SoC after RTE.	rolling_horizon_gurobi_dispatch.py:157-158
SoC update	Battery next hour = current + charge - discharge/RTE.	rolling_horizon_gurobi_dispatch.py:160-161
Revenue objective	Maximizes price x delivered power over the planning window.	rolling_horizon_gurobi_dispatch.py:163
Outputs	Direct, charge, discharge, delivered, storage, mode, objective, runtime.	rolling_horizon_gurobi_dispatch.py:170-185

If Chris asks whether rolling horizon executes one hour or one day: In the Step 2 rolling-horizon folder, each solve looks ahead H hours but executes 24 hours: `execute_len = min(24, remaining hours)` at `forecast_backtest_rolling_horizons.py:405-407`. The scenario folder executes first hour, because scenarios are built as a one-step non-anticipative controller.

8. Step 3: different scenarios folder

Purpose: instead of one forecast, build several possible wind/price futures and choose a first-hour action that performs well across those futures. This is the uncertainty-aware part.

Part	What it does	File and line
Knobs	Horizon, variants 1/3/5/7/10, storage settings, calibration mode.	different_scenarios/EXPERIMENT_KNOBS.py:11-41
Run command builder	Passes knobs into <code>run_uncertainty_aware_dispatch.py</code> .	RUN_3_SCENARIO_COMPARISON.py:67-104
Scenario MILP	Creates variables for each scenario and hour.	code/run_uncertainty_aware_dispatch.py:156-

		174
Scenario constraints	SoC, wind-only charging, charge/discharge mode, delivered, SoC update.	run_uncertainty_aware_dispatch.py:176-188
Non-anticipativity	First-hour charge/discharge/direct/mode must match across all scenarios.	run_uncertainty_aware_dispatch.py:189-195
Objective	Maximize expected scenario value, optionally blended with worst-case value.	run_uncertainty_aware_dispatch.py:197-209
First-hour execution	Only the first-hour action is executed against actual wind.	run_uncertainty_aware_dispatch.py:226-245
Single forecast comparison	Runs one forecast with recourse for comparison.	run_uncertainty_aware_dispatch.py:333-391
Scenario controller loop	Builds scenario matrices, solves scenario MILP, executes first hour, repeats.	run_uncertainty_aware_dispatch.py:394-430
Scenario parser	Reads horizon, storage, variants, calibration, output flags.	run_uncertainty_aware_dispatch.py:530-590

Why first-hour non-anticipativity matters: It prevents cheating. If Gurobi sees multiple possible futures, it cannot choose a different current action for each future. It must choose one current action now, then it can replan later.

9. Step 4: oracle upper bound folder

Purpose: show the ceiling. Oracle is the same Gurobi machinery, but planned wind and price are the actual future values. This is not deployable; it tells us how much value is theoretically available if forecasts were perfect.

Part	What it does	File and line
Knobs	Data, config, storage, horizon list, SoC fractions.	oracle upper bound/EXPERIMENT_KNOBS.py:12-36
Run command builder	Adds --oracle-only and passes the knob values.	oracle upper bound/RUN_4_ORACLE_UPPER_BOUND.py:53-89
Oracle branch	perfect_information=True uses actual_generation and actual_price for planned window.	oracle upper bound/code/forecast_backtest_rolling_horizons.py:410-412
Gurobi solver	Same solve_window structure as rolling horizon.	oracle upper bound/code/rolling_horizon_gurobi_dispatch.py:101-163
Oracle-only CSV	Filters rows where method == oracle and writes oracle_upper_bound_summary.csv.	RUN_4_ORACLE_UPPER_BOUND.py:95-105

10. B6 verification folder

Purpose: satisfy Chris's tightly scoped 2020 A/B/C x Oracle/Causal request. Do not mix this with the main paper ladder unless Chris asks specifically about B6.

Part	What it does	File and line
B6 Gurobi solver	Builds 2020 dispatch window optimization.	b6 verification/code/B6_CANONICAL_RUNNER.py:290-340
B6 constraints	Initial/terminal SoC, wind-only charge, mode, grid, SoC update.	B6_CANONICAL_RUNNER.py:323-340
B6 oracle	Uses actual wind and raw price as forecast inputs.	B6_CANONICAL_RUNNER.py:467-502
B6 causal	Uses forecast matrices, executes each window, and carries SoC.	B6_CANONICAL_RUNNER.py:505-530
B6 QA	Checks SoC bounds, simultaneous charge/discharge, wind-only charging, grid cap, revenue sum, terminal SoC.	B6_CANONICAL_RUNNER.py:421-459
B6 validator	Checks completed final package outputs.	b6 verification/code/B6_FINAL_VALIDATE.py:104-106

11. Revenue, COVE, and baseload wording

Term	What to say
Raw revenue	Dollars: sum(delivered MWh x raw LMP USD/MWh). This is used in B6 and

	scenario-style raw-LMP outputs.
Revenue metric	Normalized-price value: sum(delivered MWh x normalized price). Rolling horizon/oracle use this older project metric.
COVE	Cost divided by valued energy. In code this is effectively dispatch_cost / revenue-like value. Lower COVE is better.
Baseload	Reference case. Depending on file it may mean wind-only sell-to-grid or 100 MW constant-output storage reference. Always name which one.
Oracle	Perfect future information. Useful as upper bound, not real-life deployable.
Causal	Only uses information available at decision time, such as forecasts and current/lagged data.
Realized	Scored using what actually happened after the plan was made. Forecasts decide the plan; actual wind/price score the result.

12. Fast answers if Chris asks

Chris asks...	Answer in one sentence	Where to show
Where is the main deterministic baseline?	The deterministic paper-facing run is Step 2 rolling horizon with causal ridge forecasts and common 100 MW / 10 h CAES constraints.	rolling horizon/RUN_2_ROLLING_HORIZON.py and EXPERIMENT_KNOBS.py
Where is the optimization?	The actual Gurobi MILP is solve_window, not the RUN file.	rolling horizon/code/rolling_horizon_gurobi_dispatch.py:101-163
How does rolling horizon work?	It plans H hours, executes 24 hours, updates SoC, and solves again.	forecast_backtest_rolling_horizons.py:404-443
How do you know constraints are checked?	The realized output is checked for wind-only, grid, charge/discharge, energy availability, SoC update, and SoC bounds.	forecast_backtest_rolling_horizons.py:299-354
What makes scenario dispatch different?	The first-hour action must be identical across all future scenarios, so it is one real action, not one action per future.	run_uncertainty_aware_dispatch.py:189-195
Why is oracle not realistic?	It uses actual future wind and price inside the planning window.	forecast_backtest_rolling_horizons.py:410-412
Where are knobs changed?	Each folder has EXPERIMENT_KNOBS.py; RUN_*.py reads that file and passes flags to the worker.	All EXPERIMENT_KNOBS.py files
What is B6?	B6 is the frozen 2020 verification package with A/B/C architectures, raw realized LMP revenue, and QA outputs.	b6 verification/code/B6_CANONICAL_RUNNER.py

13. Exact commands to run

```
cd "/Users/davidvalenta/deep-wind-model-dispatch/Summer 2026 REU/100 MW baseload"
../venv/bin/python RUN_0_100MW_BASELOAD.py
cd "/Users/davidvalenta/deep-wind-model-dispatch/Summer 2026 REU/causal ridge regression"
../venv/bin/python RUN_1_FORECAST_RMSE.py
cd "/Users/davidvalenta/deep-wind-model-dispatch/Summer 2026 REU/rolling horizon"
../venv/bin/python RUN_2_ROLLING_HORIZON.py
cd "/Users/davidvalenta/deep-wind-model-dispatch/Summer 2026 REU/different scenarios"
../venv/bin/python RUN_3_SCENARIO_COMPARISON.py
cd "/Users/davidvalenta/deep-wind-model-dispatch/Summer 2026 REU/oracle upper bound"
../venv/bin/python RUN_4_ORACLE_UPPER_BOUND.py
```

14. Current repo-state note

The organized Summer 2026 REU reproducibility package is pushed at commit e184740da217ed811496c056aace5b7a61f6f860. There are unrelated unpushed local files in the working tree, mostly DAM/new-data experiments and output folders. If Chris asks for the frozen reproducible state, use the pushed commit above unless you intentionally decide to include later local experiments.